

Assignment 2: Make it fair.

Your network works! Customers are pouring in wanting to use your network! Congrats! But now your customers sometimes want to send more traffic than your links can carry. This does not happen everywhere or all the time. It happens during bursts, at busy links, and often at the worst possible moment.

In Assignment 1, your job was to make the network work. Your switches discovered the topology, learned where destinations were, forwarded packets, and routed around failed links. If a packet was lost, something had probably gone wrong. In this assignment, your network faces a different problem: **more traffic may arrive than the network can carry.**

When packets arrive faster than a link can send them, they wait in a queue. If the overload continues, the queue grows. Switches cannot hold an unlimited number of packets, so eventually they must drop some of them.

Your customers already do their part. Every sender slows down when it loses packets and speeds up again when it does not. So the network does not melt down by itself. What the senders cannot do is decide who gets the capacity. That decision is yours, and it is the subject of this assignment:

- **Part1: At one link (buffer management & scheduling):** when several customers want the same link, each must get its fair share, and a small customer must not lose packets because a large one filled the buffer.
- **Part2: Across the network (traffic engineering):** when one link is full and another is idle, the capacity exists, but on a different path. Only your routing can reach it. To place each customer on a path so that the whole network's capacity is used is called traffic engineering.

Protect your own control packets. Your HELLO packets use the same links and queues as customer traffic. If a full queue drops them, your switch can decide that a healthy link has failed. **No links fail in this assignment**, so any link-down declaration is a false alarm, and it fails a floor.

Your switch controls three things in A2:

1. **The queue:** how much buffer space each customer may use, and when a packet is dropped.
2. **The schedule:** which waiting packet is sent next, and how a link's capacity is divided among customers.
3. **The path:** which route each customer's packets take. Shortest-path is a choice, not a default you are stuck with.

The choices you make here will also become part of the network you carry into future assignments.

The Rules of the Machine

Good design starts with knowing what you cannot change.

Here is what is fixed:

	Every switch in every world
Buffer	40,000 bytes for each switch
Ports	At most 5 on a switch
Link Delay	0.5ms on every link
Link Capacity	Differs from link to link; your switch is told the capacity of each of its links at the start.

The buffer is the resource you must ration: 40,000 bytes hold 40 full-size (1,000-byte) packets. Every switch gets the same amount: a switch with more ports, or in a busier place, gets no more. ***How you divide it is your choice: per port, per customer, or one shared pool.*** You set queue sizes and drop lines in full-size packets. The switch counts them in bytes. So a queue of 4 packets holds 4,000 bytes: 4 full-size packets, or 20 of the 200-byte packets that Jane Street and Fortnite send. Acks and your own control packets are smaller still.

What you may do, and what you must do.

- **You may drop early.** To drop a packet before the buffer is full is a valid design.
- **You may serve your control packets first.** HELLOs, routing updates and load reports can go ahead of customer data, as in real networks. They are small, and you may send them freely.
- **You must declare a link state.** Call `link_down(port)` when your control plane decides that a link is dead, and `link_up(port)` when it is alive again. We compare these calls with the truth. No link fails in this assignment, so a correct network never calls `link_down`.

The Rules of the World: Your Customers

Your customers run different applications, and they ask for different amounts. Your switch is never told which application a customer runs, and it does not need to know. All it sees is whose

packet this is, from the source prefix. Every customer is held to the same standard: its fair share of the network.

A **customer's fair share** is the rate that it asks for, or an equal split of the link that it shares with others, whichever is smaller. A small customer gets all that it asks for, and the large customers divide what is left. The table uses the term in this sense, and Part 1 gives the full rule.

Application	Traffic pattern	Asks for (example)	What it needs from you
Netflix (video streaming)	steady	3-5 Mbit/s	its fair share. <i>(Important to not throw remote at TV)</i>
Amazon (web store)	on for 8-10 s, then off for 8-10 s	6-8 Mbit/s while on	its fair share soon after a burst starts. <i>(Flash sale events)</i>
Steam (game downloads)	continuous	no limit	all that is left over. <i>(Haul games in volume)</i>
JaneStreet (high-frequency trading)	steady, small (200-byte) packets	0.05-0.2 Mbit/s	all it asks for, with low delay. <i>(Fin bros care about this a lot)</i>
Fortnite (online game)	steady, small (200-byte) packets, both directions	0.3-0.6 Mbit/s each way	all it asks for, with low delay, both ways. <i>(True gamers know this pain)</i>

The "asks for" column is an example, to show the scale. The real rates change from world to world and grow with the load. Your switch is told the exact rate of each customer at the start (see "The customer contract" below).

This scaffolding is for this assignment only: every customer asks for exactly the rate that your switch is given, and no link fails. In Assignment 3 your switch is told nothing about its customers. It must measure what each one sends, and links fail while it does.

These customers pull against each other. The capacity and buffer that you hold for one are not there for another: hold some for Amazon while it is quiet and you waste it; give it all to Steam and Amazon's next burst starts with nothing. A design that pushes hard for one floor can fail another, and where you settle is your design. Wherever you settle, you may hold some customers back. What happens next depends on how that customer slows down, and they do not all slow down by the same rule.

How Customers react to Congestion

There are two rules, and a world may run either one on any customer.

AIMD (additive increase, multiplicative decrease) halves its rate when it notices a lost packet, and adds a small increment back for every 100 ms without a loss: a sharp cut, then a slow climb, as real TCP senders do.

Rate matching sends at the rate that was acknowledged over the last 100 ms or so, plus a quarter more to find spare room: no sharp cut, in the spirit of Google's BBR ([read about BBR](#)). One dropped packet changes almost nothing. Its rate follows the rate at which your switch serves it. It also limits the data that it has in flight to 2 BDP (BDP is the bandwidth-delay product; we define it in lecture on Tuesday). One BDP fills the path, and the rest waits in the queue of the link that limits it. So a rate-matching customer keeps a standing queue of about 1 BDP.

Both send a lost packet again. A copy of a packet that already arrived counts for nothing. The exact rules are in the source you are given, in *src/flow.rs*.

Do not put the two kinds in the same queue. An AIMD sender halves its rate at a drop. A rate-matching sender does not, so in a shared queue the AIMD sender gets less and less of the link. The customer contract tells you which rule each customer follows. What each rule does to its own queue is for you to work out: read the two rules, run a practice world, and see what each kind does when you drop its packet and when you serve its queue slowly.

This matters because every packet that waits anywhere in your switch uses the same 40 packets of buffer. You trade three things against each other:

- **Queue assignment.** A queue for each customer isolates it, and the scheduler sets its share. In a shared queue, one customer can fill it and the packets of the others are dropped. But each queue that you add leaves less buffer for the others: 5 ports with 4 customers on each gives 2 packets for each queue.

- **Buffer allocation.** If your queue limits add up to 40 packets or less, each queue always has its space, and that space is idle when the queue is empty. If they add up to more, a busy queue can take the space that a quiet queue needs later.
- **Queue depth.** A deep queue absorbs a burst without a drop, but every packet in it waits longer. A shallow queue keeps the queueing delay low, and drops sooner.

Make each decision for each kind of sender, and give your reasons on the design card. In this assignment you design for behaviour that you know: there are only two rules, both are in the source, and your switch is told which one each customer follows. In Assignment 3 it is not told. Your switch will have to watch each customer's traffic, decide from that how the customer behaves, and choose its queue while it is still not sure.

Customer Contract

Your switch does not have to work out which rule a customer follows, or how much it wants. At the start, every switch is given the same contract. For each customer it gives the source prefix, the rule (AIMD or rate matching) and the rate that the customer asks for (no limit for Steam). The contract is what the customer bought, not a measurement, and in this assignment every customer keeps it. It does not say when an Amazon burst begins.

What your switch can see. About the load, your switch knows three things: the contract, the capacity of each of its links (told once, at the start), and how many packets each port dropped, queue by queue (told each time its timer fires). The drops show where a link is full and when the load changes. You do not have to measure sending rates, queue lengths or delay. That comes in Assignment 3. The kit's README gives the exact fields, and *examples/load_report* reads all three.

Part 1: Welcome to Rush Hour

Critical warning: link overload! The combined demand on one link is more than the link can carry. Your task has two parts. **Buffer management:** how much of the switch's buffer each queue may use, and when a packet is dropped. **Scheduling:** which queue sends the next packet.

There is no single correct design. A deep queue loses fewer packets but makes everyone wait; a shallow one is quick but wastes capacity. Strict shares protect small customers; looser ones carry more in total. No one goal may be bought at the complete expense of the others. That is what the floors below enforce. Above them, the **trade-offs are yours: they belong on your design card, and the quiz will ask you to defend them.**

What "fair" means here. Each customer's fair share is what it asks for, or an equal split of the bottleneck, whichever is smaller. Small customers get everything they want; big customers split what is left. This is max-min fairness, computed once per customer across its whole path.

You pass if your network clears the following floors. A floor does not rank your design, it catches a disaster. There are three: the first two are about how much your customers receive, and the third is about your own control traffic. Each is checked in every graded one-second window.

- **(F1) Per-customer throughput:** Each customer receives at least 40% of its max-min fair share. We check every customer, not the average, so one starved customer fails the floor.
- **(F2) Total throughput:** All customers together receive at least 55% of the total that the max-min fair allocation delivers. F1 alone would pass a network that holds every customer at 40% and leaves the link half idle. F2 catches a design that drops too early and wastes capacity.
- **(F3) No false link-down:** Your switches never declare a link down. No link fails in this assignment, so every such declaration is false. To route around a congested link is correct. To declare a healthy link dead is a bug.

Not graded: the two seconds after each Amazon burst begins, and a second in which a burst was on for only a part.

Loss and delay are on your report card, but they are not graded in this assignment. Read them to see what your design costs: the card gives the 99th percentile of the delay of each customer, and a small customer whose packets wait is waiting behind the packets of another. The Assignment 1 rules never expire: no forwarding loops, and no customer left with nothing delivered. The 99.9% delivery gate is gone; loss is legal now.

Part 2: Take the Detour

In Part 1, rush hour hit one bottleneck. Now several links are hit, and you need to route customers smartly to ensure throughput and fairness. In Part 2, the path you choose decides whether you pass. Picture rush hour on a highway with a side road beside it. If every car takes the highway, it jams while the side road sits empty. Deciding which traffic takes which road, so that no road jams while another sits empty, is called **traffic engineering**, and it is what Part 2 asks of you.

We fix where each customer connects. You decide the path that its traffic takes. So traffic engineering is two things that you design: the set of paths that your network uses, and which customer's traffic goes on which path. You need more than one path because the shortest paths cannot carry the load. You need to choose each customer's path with care because we measure the customer against the best path that the network has, not against the path that you chose.

- **Multipath routing.** Shortest-path routing alone cannot carry the load: the thin links are too small, and no queue or drop policy fixes that. The capacity exists only across more than one path.
- **Fair share over available paths.** A customer's fair share is computed over every route at most two hops longer than its shortest, not only the route you chose. Example: a customer left on a 2 Mbit/s link with three others gets about 0.5 Mbit/s. If an idle detour could have carried 5 Mbit/s, its fair share is 5, it needs 2 (40%), and it fails the per-customer throughput floor.

What you decide. Traffic engineering is a loop that your network runs for the whole run: learn where the load is, choose a path for each customer, and change the paths when the load changes. That gives you three design decisions, and each has a trade-off:

1. **Load information.** A switch that decides alone acts at once, but it sees only its own links, so it can move traffic to a detour that is already full two hops away. Switches that share one picture of the load can find the path that is really idle. But that picture costs control packets, and it is always a little old.
2. **Path selection.** Keep each customer on one path at a time. Real routers do this, because packets that arrive out of order look like loss to a sender. So you assign whole customers, and they are not all the same size: to move a large customer frees the most capacity, but the detour must have that much room. A longer detour gives you more capacity to choose from, but it loads more links, and it can take capacity from customers who had no congestion.
3. **Route changes.** Only Amazon changes its demand during a run, so the load changes when a burst starts or stops, every 8–10 seconds. The two seconds after a burst begins are not graded. React later than that, and a link stays congested in graded seconds. React to every report, and the traffic that you moved makes the new path look full, so it moves back. Each action lands 2 ms after you send it, so for a short time old and new routes are mixed. If they point at each other, packets loop, and a loop fails the run.

You pass if the same three floors from Part 1 hold, with the same numbers. Only two things differ: fair share is computed over the available paths, and grading starts three seconds in, so you have time to place your traffic.

*(Next assignment, links start failing again, while all of this is still happening. Loss will stop having a single explanation, and one of your customers will stop playing by the rules. Your network will need to see what is happening before it can act on it. **First, survive rush hour.**)*

What you get

Everything from A1: the simulator, the Rust SDK, and your own A1 switch, which is now your starting point. Keep your A1 kit at hand. Its worlds have no congestion, so they are the calm place to test: whenever you change your switch, run it there first and confirm it still discovers the

network, forwards, and routes around failures. If it breaks there, the bug is in your routing, not in how you handle load. ***This is by design: each assignment's simulator removes one protection the last one gave you, so you can try a design decision in the calmest world where it matters, and move it up only once it holds. => A1 has no congestion; A2 has congestion but no failures; A3 brings failures back.....and so on.***

New in this kit:

1. **The customer application source code:** the same adaptive apps grading uses, both sender rules included. Watch how they respond to your drops; that feedback loop is the assignment.
2. **5 Practice worlds:** three with one bottleneck and two with several; the exact scoring code, and unlimited runs.
3. **A load dial:** --load-scale reruns any world with every customer asking for more or less. We grade every world twice, at $\times 1$ and at $\times 1.5$. Practice across $0.5\times$ to $2\times$ to see where your design bends.

Using AI

AI knows every scheduling algorithm in the textbook and will happily hand you a fancy queuing algo in thirty seconds. What it doesn't know or is unable(not yet) to understand and reason cleanly about is what your world punishes: our buffer budget, our burst shapes, our topologies, our two sender rules. The AI can write your scheduler and drop policy; ***it cannot sit your quiz, and help you reason about the performance of the code you submitted.***

Grading

- To pass A2, your network must clear all three floors. We run it on hidden worlds, each at $\times 1$ and $\times 1.5$ load, and a run counts only when every floor holds.
- Graded attempts: you get three graded attempts; the best one counts. I will release server login details on Oct 7th or before; you can log in and attempt your graded attempt anytime until the deadline and get your result in a few minutes.
- **Due Date: Oct 16th Eod**